Agent 开发面试知识手册

Python LangChain LangGraph RAG MCP 工程实践

适用于 AI Agent 开发 大模型应用开发 Python 后端岗位

沈鑫 面试复习版

更新日期 2026 年 9 月

使用方法

这份手册面向 Agent 开发岗位的技术面试。每道题先给出可在 20 至 40 秒内说清的核心回答，再补充工程细节和项目落地方式。复习时先掌握定义、机制、权衡、落地四步回答法，再结合自己的项目数据补充结果。

四步回答法

- 定义：先用一句话说明概念解决什么问题。
- 机制：说明数据如何流动、状态如何变化、失败如何处理。
- 权衡：指出准确率、延迟、成本、可控性和安全性的取舍。
- 落地：用项目中的模块、技术选型、故障处理或测试结果收尾。

目录

1. 一 Agent 基础与执行循环
2. 二 提示词 工具调用与结构化输出
3. 三 LangChain 与 LangGraph
4. 四 RAG 检索与答案溯源
5. 五 记忆 上下文与会话状态
6. 六 异步并发 流式响应与可靠性
7. 七 MCP 与多 Agent 协作
8. 八 测试 评估与可观测性
9. 九 安全 权限与数据隔离
10. 十 系统设计题
11. 十一 项目深挖回答模板
12. 十二 高频速记清单与参考资料

一 Agent 基础与执行循环

先说明 Agent 的边界，再解释模型、工具、状态和终止条件如何组成闭环。

Q1 什么是 AI Agent，它与普通聊天机器人有什么区别?

核心回答：Agent 是由模型驱动、能感知上下文、选择工具、执行动作并根据结果继续决策的应用。普通聊天机器人通常只生成一次回复，Agent 则包含可重复执行的决策和动作闭环。

- 核心组件包括模型、指令、工具、状态或记忆、控制流和运行时。
- 是否称为 Agent 不取决于使用了哪个框架，而取决于系统是否让模型参与下一步动作决策。

项目落地：可以用职位匹配项目举例：模型先判断是否检索岗位，再调用匹配或技能差距工具，读取结果后决定继续检索还是生成报告。

Q2 Agent 与 Workflow 应该如何选择?

核心回答：步骤固定、规则清楚、可预测性优先时用 Workflow；路径依赖输入、工具结果或环境反馈时才引入 Agent。生产系统常把二者组合：确定性流程负责关键边界，模型只负责需要语义判断的节点。

- Workflow 更易测试、审计和控制成本。
- Agent 灵活，但会增加路径不确定性、调用次数和安全风险。

Q3 典型 Agent Loop 是什么?

核心回答：典型循环是模型读取当前状态，决定直接回答或发起工具调用；运行时校验参数并执行工具；结果写回状态；模型再次判断，直到完成、达到步数上限、超时或触发人工确认。

- 循环必须有最大步数、总时限、成本预算和重复调用检测。
- 运行时应把模型决策与工具执行分开，模型不能直接获得底层执行权限。

项目落地：MiniOpenClaw 中可以按 Model Call、Tool Dispatch、Observation、Next Step、Final Answer 描述闭环。

Q4 什么是 ReAct?

核心回答：ReAct 将推理与动作交替组织。模型根据任务形成下一步意图，调用环境或知识工具获取观察结果，再更新计划并继续。它适合信息不完整、需要多步交互的任务。

- 优点是能利用外部信息修正路径，执行轨迹也便于调试。
- 风险是循环过长、错误累积和思考内容泄露。工程上应记录可审计的动作摘要，不依赖向用户暴露内部推理文本。

Q5 如何做任务规划与分解?

核心回答：先把目标转成可验证的子任务和依赖关系，再决定串行、并行或条件执行。规划结果应包含输入、输出、使用工具、完成标准和失败策略。

- 简单任务不需要先生成完整计划，边执行边规划通常更省延迟。
- 高成本或高风险任务适合 plan-and-execute，并在执行后校验计划是否仍然成立。

Q6 Agent 如何判断停止?

核心回答：停止条件应由业务完成条件和运行时护栏共同决定，包括模型返回最终结果、必需字段齐全、目标状态达成、达到最大步数、超时、预算耗尽、连续重复动作或人工拒绝。

- 只依赖模型说完成会产生死循环或过早结束。
- 终止原因应进入结构化日志，方便区分正常完成、降级完成和异常终止。

二 提示词 工具调用与结构化输出

重点不是提示词技巧，而是让模型输入、工具契约和输出校验形成稳定接口。

Q7 什么是 Tool Calling?

核心回答：Tool Calling 是模型根据工具名称、说明和参数模式生成结构化调用请求，由应用程序执行工具，再把结果作为工具消息交回模型。模型负责选择，运行时负责验证、授权和执行。

- 工具调用不等于模型直接执行代码。
- 工具定义应使用清晰名称、窄参数、明确单位、边界条件和错误语义。

Q8 如何设计一个高质量工具?

核心回答：一个高质量工具只承担单一职责，输入模式明确，输出稳定，副作用可识别，并能返回可重试错误与不可重试错误。说明文字要告诉模型何时用、何时不要用。

- 避免万能 search_or_write 工具，也不要让参数同时承载自然语言和多个隐式操作。
- 返回结果应控制长度，并携带来源、对象标识、状态码和下一步可用信息。

Q9 结构化输出有什么价值?

核心回答：结构化输出把自然语言约束成 JSON、Pydantic 模型或数据类，便于后端校验、入库、渲染和下游调用。它降低了解析歧义，但不能替代业务校验。

- 需要校验枚举、范围、必填字段、跨字段约束和语义合理性。
- 解析失败时应有限次重试，并把验证错误反馈给模型，而不是静默吞掉。

项目落地：职位匹配结果可固定为 job_id、score、matched_skills、missing_skills、evidence 等字段。

Q10 如何处理工具参数错误?

核心回答：运行时先做模式校验和业务校验。缺少可补充参数时返回结构化错误让模型修正；涉及权限、资源不存在或危险操作时直接拒绝或转人工。重试必须有次数上限。

- 不要把完整异常堆栈交给模型或用户。
- 错误对象至少包含 error_type、message、retryable 和 safe_details。

Q11 什么时候可以并行调用工具?

核心回答：多个工具调用互不依赖、没有资源冲突且都为只读或幂等操作时可以并行。后续调用依赖前一步结果、存在写冲突或顺序语义时必须串行。

- 并行前要设置并发上限、超时和取消传播。
- 结果合并需要稳定顺序或显式标识，不能依赖任务完成先后。

Q12 如何保证有副作用工具的幂等性?

核心回答：客户端为一次业务操作生成幂等键，服务端记录键与结果；重复请求返回原结果，不重复扣款、发信或写入。幂等键应与用户、操作类型和有效期绑定。

- 重试机制和幂等性要同时设计。
- 数据库唯一约束、事务和状态机比只在内存里去重更可靠。

Q13 System Prompt 应该包含什么?

核心回答：System Prompt 应明确角色、目标、可用能力、必须遵守的边界、工具使用原则、输出格式和无法完成时的行为。稳定规则放系统层，用户数据和检索内容作为不可信数据单独传入。

- 避免塞入大量互相重复的规则。
- 安全边界应由代码和权限系统强制执行，不能只靠提示词。

三 LangChain 与 LangGraph

回答框架题时要讲抽象层次、状态模型和工程选择，不要只罗列 API 名称。

组件	定位	适用场景
LangChain	模型、工具、Agent 等高层抽象与集成	快速搭建常见工具调用与 RAG 应用
LangGraph	面向长流程和有状态 Agent 的编排运行时	分支、循环、持久化、人工确认、恢复和流式执行
自定义代码	直接实现状态机和运行时	约束简单、依赖少或需要完全控制底层行为

Q14 LangChain 与 LangGraph 有什么区别?

核心回答：LangChain 提供模型、消息、工具、检索和 Agent 等较高层抽象；LangGraph 更接近有状态编排运行时，用节点、边和状态表达长流程。需要精细控制、持久化、人工确认和故障恢复时优先考虑 LangGraph。

- 二者不是互斥关系，LangGraph 可以复用 LangChain 的模型和工具组件。
- 简单的一次工具调用流程不必为了框架完整性强行上图编排。

Q15 LangGraph 的 State Node Edge 分别是什么?

核心回答：State 是跨步骤共享的数据结构；Node 是读取状态并返回增量更新的计算单元；Edge 决定执行顺序，Conditional Edge 根据状态选择下一节点。编译后的图负责调度、合并状态和持久化。

- 状态字段要区分覆盖、追加和自定义归并语义。
- 节点应尽量保持输入输出清晰，副作用集中管理。

Q16 为什么状态更新要使用 Reducer?

核心回答：当多个节点更新同一字段时，Reducer 定义如何合并增量，例如消息列表追加、集合去重或分数取最大值。没有明确归并规则时，并行分支可能互相覆盖或产生不确定结果。

- Reducer 应满足可预测性，最好具备结合性；需要重放时还应避免依赖当前时间或随机顺序。

Q17 Checkpoint 与 Thread 是什么关系?

核心回答：Checkpoint 保存某个执行步骤后的状态快照，Thread 标识一条连续会话或任务轨迹。用同一 thread_id 调用时，运行时可以读取历史检查点并继续执行。

- 生产环境要使用持久化存储，内存检查点只适合测试。
- 检查点中不要无边界保存大文件和全部工具原始输出。

项目落地：MiniOpenClaw 的 JSONL 会话日志可以解释为事件记录；若升级为图运行时，可把可恢复状态单独存入检查点。

Q18 LangGraph 如何实现 Human in the Loop?

核心回答：在高风险工具前触发 interrupt，把当前状态持久化并返回待审批信息；用户选择批准、修改或拒绝后，用同一线程恢复执行。审批对象应包含工具名、参数、影响范围和风险说明。

- 中断前必须完成 checkpoint，否则进程退出后无法恢复。
- 审批应绑定具体调用和状态版本，防止旧审批被重放到新参数。

Q19 如何保证 Durable Execution?

核心回答：把每个步骤的输入、状态更新和副作用结果持久化，在失败后从最近的成功检查点恢复。可重放节点必须确定性强；带副作用操作需要幂等键或把副作用结果写入状态。

- 随机数、当前时间和外部 API 结果会破坏重放一致性，应显式保存。
- 恢复时要区分重新计算和复用已完成结果。

Q20 如何设计 Agent 中间件或生命周期 Hook?

核心回答：中间件适合统一处理模型调用前后的上下文、日志、预算、重试、安全检查和工具审批。Hook 不应隐藏关键业务分支，否则调试时难以还原真实路径。

- 每个 Hook 要定义触发时机、可修改对象和异常传播策略。
- 观测 Hook 与业务 Hook 分开，避免日志故障影响主流程。

项目落地：可结合 MiniOpenClaw 的生命周期 Hook，说明如何统计模型耗时、工具耗时、token 和异常类型。

四 RAG 检索与答案溯源

面试官通常会从切分、召回、重排、生成、评估五个环节连续追问。

架构	特点	适用场景
两步 RAG	先检索再生成，路径固定，延迟可预测	企业知识问答、FAQ
Agentic RAG	模型决定何时检索及调用哪个来源	多来源研究、复杂任务
Hybrid RAG	固定检索结合改写、校验和必要的循环	需要质量控制的领域问答

Q21 RAG 的完整链路是什么?

核心回答：离线链路包括数据清洗、切分、元数据构建、向量化和索引；在线链路包括查询理解、召回、融合、重排、上下文组装、生成和引用。每个环节都应单独可评估。

- RAG 解决的是外部知识接入和答案依据问题，不保证模型一定忠实使用证据。
- 知识更新时要处理增量索引、删除、版本和权限同步。

Q22 Chunk 应该如何切分?

核心回答：切分目标是让每个 Chunk 语义完整、可独立检索并能放入上下文。优先按标题和自然段切分，再设置长度上限与适量重叠；表格、代码和列表需要专门规则。

- Chunk 太小会丢上下文，太大则降低召回精度并浪费 token。
- 元数据至少保留文档、章节、页码或位置、权限标签和版本。

项目落地：职位匹配项目可说明采用结构化字段、规范化原文和标题感知 Chunk，使技能名称精确匹配，同时保留职责语境。

Q23 向量检索和关键词检索有什么区别?

核心回答：向量检索擅长语义相似和改写表达，关键词检索擅长精确术语、编号、专有名词和稀有词。工程上常使用混合召回，再通过融合或重排得到候选。

- 向量相似不等于事实相关。
- 岗位技能、证书名、版本号等字段往往需要 BM25 或过滤条件辅助。

Q24 RRF 是什么?

核心回答：RRF 是 Reciprocal Rank Fusion。它不直接比较不同检索器的原始分数，而是按候选在各列表中的名次累加 1/(k + rank)。这样能融合分数量纲不同的结果。

- k 用来降低头部名次差异的敏感度。
- RRF 简单稳定，但不会理解内容相关性，因此通常还会接重排模型或业务规则。

项目落地：职位匹配项目中可解释为先融合全文检索和向量召回，再重排兼顾精确技能词与语义改写。

Q25 召回和重排分别解决什么问题?

核心回答：召回从大规模语料中快速找出高覆盖候选，目标偏 Recall；重排在较小候选集上使用更强模型或交叉编码器，提高前几名的相关性，目标偏 Precision 和 NDCG。

- 候选过少会让重排无米下锅，候选过多会增加延迟。
- 应分别记录召回耗时、重排耗时、命中率和最终上下文长度。

Q26 查询改写有哪些常见方法?

核心回答：常见方法包括去歧义、补全上下文、生成多个查询、拆分子问题和把自然语言转成结构化过滤条件。改写后要保留原始意图，避免凭空加入实体或限制条件。

- 多查询能提高召回，但会增加成本并带来重复结果。
- 对短问题可结合会话历史生成独立可检索问题。

Q27 如何生成可验证引用?

核心回答：每个检索片段携带稳定 source_id、位置和版本；生成时要求答案中的关键断言关联证据；输出后检查引用是否存在、是否支持对应断言，并允许用户回到原文。

- 引用存在不代表引用正确，需要做 entailment 或人工抽检。
- 不要让模型自行编造 URL、页码或文档标识。

项目落地：可以强调项目中让生成材料携带可验证 Citation，使能力描述能回溯到用户原始资料。

Q28 如何评估 RAG?

核心回答：分层评估检索和生成。检索关注 Recall@k、MRR、NDCG 和命中率；生成关注答案正确性、完整性、忠实度、引用准确率和拒答质量；系统还要看延迟、token 成本与失败率。

- 先建立带问题、参考答案和相关文档的测试集。
- 线上失败样本应回流到离线回归集，避免只看主观演示效果。

五 记忆 上下文与会话状态

记忆不是把所有历史都塞进提示词，而是决定保存什么、何时读取、何时遗忘。

Q29 短期记忆与长期记忆有什么区别?

核心回答：短期记忆服务于当前线程，常包含消息历史、当前计划和临时结果；长期记忆跨线程保存稳定事实、用户偏好或已验证经验。二者应使用不同作用域、生命周期和权限。

- 短期状态通常由 checkpoint 管理。
- 长期记忆需要写入策略、冲突解决、过期和删除机制。

Q30 为什么长对话不能一直保留全部消息?

核心回答：完整历史会占用上下文窗口、增加成本和延迟，旧信息还可能干扰当前决策。常见策略是保留近期窗口，压缩较早消息，并把稳定事实提取到长期存储。

- 压缩触发条件可以结合消息数、token 数和任务阶段。
- 工具大结果应保存引用或摘要，而不是反复放入上下文。

项目落地：MiniOpenClaw 中可说明采用消息数与 token 双阈值触发压缩，并保留系统约束、未完成任务和关键工具结果。

Q31 上下文摘要有什么风险?

核心回答：摘要可能遗漏约束、混淆说话人、强化错误信息或在多次压缩后发生语义漂移。应保留关键原文引用、结构化事实和摘要版本，并对高风险字段禁止只存摘要。

- 可以定期从原始事件重新生成摘要。
- 压缩前后用测试问题检查任务状态和关键事实是否仍可恢复。

Q32 长期记忆应该何时写入?

核心回答：只写入未来仍有价值、来源明确且用户有权保存的信息。可以在交互热路径同步写入关键事实，也可以后台提取候选记忆后去重、验证再落库。

- 不要把模型猜测当作用户事实。
- 记忆记录应包含来源、时间、置信度、作用域和可删除标识。

Q33 如何设计会话隔离?

核心回答：用租户、用户、渠道、会话和话题组成复合命名空间，所有读取和写入都必须带作用域条件。缓存键、向量库过滤、数据库行级权限和日志也要使用同一隔离规则。

- 只在提示词里写用户 ID 不能形成安全隔离。
- 跨会话共享记忆必须显式授权并记录来源。

项目落地：飞书接入中可把频道与话题标识映射成稳定 session_key，避免不同群聊或话题串线。

六 异步并发 流式响应与可靠性

这部分考察 Agent 是否能从演示代码升级为可运行的后端服务。

Q34 asyncio 适合什么场景?

核心回答：asyncio 适合大量 I/O 等待，例如模型请求、数据库查询和网络工具调用。CPU 密集任务会阻塞事件循环，应放入进程池、线程池或独立服务。

- 协程不是自动并行；只有在 await 让出控制权时其他任务才运行。
- 不要在 async 函数里直接调用长时间阻塞的同步库。

Q35 gather 与 TaskGroup 有什么区别?

核心回答：gather 适合收集一组已知 awaitable 的结果；TaskGroup 提供结构化并发，在作用域退出前等待所有子任务，并对异常和取消进行统一管理。新代码通常优先使用 TaskGroup 表达同生共死的任务组。

- 如果部分失败可以接受，要显式定义结果模型和降级策略。
- 不要吞掉 CancelledError，否则超时和关闭流程可能失效。

Q36 如何限制工具并发并实现背压?

核心回答：入口使用有界队列限制待处理任务，执行层用 Semaphore 限制模型或工具并发。队列满时应拒绝、排队或降级，而不是无限创建任务。

- 并发上限应按供应商限流、连接池和下游容量设置。
- 监控队列长度、等待时间、执行时间和被拒绝数量。

Q37 SSE 与 WebSocket 如何选择?

核心回答：SSE 基于 HTTP 单向推送，适合服务器持续向浏览器发送 token 和状态事件，重连和代理兼容性较好；WebSocket 支持全双工，适合高频双向交互。普通文本流式回答通常 SSE 足够。

- SSE 事件建议包含 event、id、data，并发送心跳保持连接。
- 客户端断开后服务端要取消下游模型和工具任务，避免继续计费。

项目落地：MiniOpenClaw 可说明将模型 token、工具开始、工具结果、错误和最终完成统一编码成 SSE 事件。

Q38 超时应该如何分层?

核心回答：至少设置连接超时、单次读取超时、工具总超时、模型调用超时和整个 Agent 运行超时。上层超时应向下传播取消，并在日志中记录到底是哪一层超时。

- 超时后不能盲目重试有副作用操作。
- 可以为检索和低优先级工具设置更短预算，为人工确认单独使用可持久化等待。

Q39 重试策略如何设计?

核心回答：只对临时性错误重试，例如网络抖动、限流和部分 5xx；使用指数退避、随机抖动和最大次数。参数错误、权限错误和确定性的业务失败不应重试。

- 重试前判断请求是否幂等。
- 遵守 Retry-After，并把多级重试预算统一管理，避免重试风暴。

Q40 JSONL 会话日志有什么优缺点?

核心回答：JSONL 追加写简单、易调试、便于逐条恢复，适合单机原型和审计事件。缺点是并发写、随机查询、索引、事务和压缩能力有限，规模增大后应迁移到数据库或日志系统。

- 每条事件应包含 event_id、session_id、sequence、timestamp、type、payload_version。
- 恢复时先校验顺序和完整性，再重建派生状态。

项目落地：可以结合 MiniOpenClaw 说明会话持久化和分叉树结构，再补充生产化时使用数据库及乐观锁。

七 MCP 与多 Agent 协作

MCP 是工具和上下文接入协议，多 Agent 是任务组织方式，两者解决的问题不同。

Q41 MCP 是什么?

核心回答：Model Context Protocol 是连接 LLM 应用与外部数据和能力的开放协议。它基于 JSON-RPC，通过 Host、Client、Server 分工和能力协商，让不同工具与上下文来源以统一方式接入。

- Host 管理用户授权、模型交互和多个 Client。
- 每个 Client 与一个 Server 建立隔离连接，Server 提供专门能力。

Q42 MCP 中 Prompts Resources Tools 有什么区别?

核心回答：Prompts 是用户选择的模板化交互；Resources 是由应用读取并放入上下文的数据；Tools 是模型可以请求执行的函数。三者的控制主体和安全边界不同。

- Resources 适合文件、文档和只读上下文。
- Tools 可能产生副作用，应进行参数校验、授权和人工确认。

Q43 什么是能力协商?

核心回答：初始化时客户端和服务端声明各自支持的协议能力，后续只能使用双方已协商的功能。这样可以支持渐进扩展和版本兼容，避免一方假设对方支持不存在的特性。

- 能力声明不等于权限授权。
- 版本、能力和用户权限应分别校验。

Q44 如何保护 MCP 工具安全?

核心回答：Host 对每个 Server 实施最小权限和数据隔离；工具调用前展示清晰参数和影响范围；服务端仍要执行身份认证、授权、输入校验、限流和审计。工具说明与外部内容都应视为不可信输入。

- 不要把整个会话无条件发送给所有 Server。
- 本地 Server 也可能执行危险代码，不能因为本地运行就默认可信。

Q45 什么时候需要多 Agent?

核心回答：当任务存在明显的专业边界、可并行子任务、不同权限域或独立上下文时，多 Agent 才有价值。协调 Agent 负责拆解与合并，专业 Agent 负责各自受限领域。

- 多 Agent 会增加通信成本、状态一致性和错误定位难度。
- 如果只是几个固定步骤，用普通函数或图节点更简单。

Q46 多 Agent 如何避免相互污染和重复工作?

核心回答：为每个 Agent 定义输入输出契约、可见上下文和可用工具；任务分配携带唯一 ID 和完成标准；共享状态采用版本控制或单一写入者；协调层检测重复任务与冲突结果。

- 不要让所有 Agent 共享完整历史和全部工具。

- 合并结果时保留来源，并对关键冲突触发验证或人工决策。

八 测试 评估与可观测性

Agent 的非确定性要求同时测试代码正确性、模型行为和完整执行轨迹。

Q47 Agent 测试应该如何分层?

核心回答：单元测试验证解析器、状态更新、权限和工具函数；集成测试验证模型适配器、数据库与外部工具；端到端评估验证真实任务完成度；线上监控发现分布漂移和新失败模式。

- 单元测试中用 Fake Model 或固定响应保证确定性。
- 外部服务通过录制响应、测试容器或契约测试隔离。

项目落地：可以介绍用 Pytest 覆盖 SSE、工具闭环、上下文压缩、飞书更新和并发执行。

Q48 Agent 常见的三类评估是什么?

核心回答：最终回答评估任务结果；单步评估某次工具选择和参数；轨迹评估完整工具调用序列与决策路径。复杂 Agent 通常需要三类结合。

- 固定流程可做严格轨迹匹配。
- 允许多条正确路径时，应评估必需工具、禁止工具、参数质量和总体效率。

Q49 LLM as Judge 有什么风险?

核心回答：它适合评估开放式质量，但可能受提示方式、位置偏差、模型偏好和被评内容中的注入文本影响。应使用明确量表、少量人工校准、盲评或成对比较，并报告一致性。

- 能用代码规则判断的格式、引用存在性和权限问题不要交给 Judge。
- 评估模型与被评模型最好不要完全同源且未经校准。

Q50 如何建立回归评估集?

核心回答：从真实用户任务、历史故障、边界条件和安全对抗样本中构建数据集，记录输入、期望结果、允许轨迹和禁止行为。每次修改模型、提示词、工具或检索索引后执行对比。

- 评估集要覆盖常见任务和高风险长尾，而不是只收集容易成功的样本。
- 线上新失败样本经脱敏和审核后回流。

Q51 Agent 需要记录哪些可观测数据?

核心回答：一次运行应能还原模型调用、工具调用、状态转移和最终结果。关键字段包括 trace_id、thread_id、步骤、输入摘要、工具参数摘要、耗时、token、成本、错误和终止原因。

- 敏感数据应脱敏或只记录哈希和引用。

- 日志、指标和 trace 要能通过统一 ID 关联。

Q52 哪些指标适合衡量生产 Agent?

核心回答：质量指标包括任务成功率、人工接管率、引用正确率和无效工具率；性能指标包括首 token 延迟、总延迟和队列时间；成本指标包括每任务 token 与工具成本；可靠性指标包括超时、重试和恢复成功率。

- 平均值会掩盖长尾，应查看 P50、P95 和 P99。
- 业务成功率要与模型评分分开定义。

九 安全 权限与数据隔离

安全题的关键是分层防护：输入不可信、模型不可信、工具有权限边界、动作可审计。

Q53 什么是 Prompt Injection?

核心回答：Prompt Injection 是不可信输入诱导模型忽略原有目标或泄露信息。它既可能来自用户，也可能藏在网页、文档、工具结果和其他 Agent 消息中。

- 提示词不能从根本上消除注入。
- 防护需要内容分区、最小权限、工具白名单、参数验证、输出检查和高风险操作确认。

项目落地：职位匹配知识库可说明检索前扫描可疑指令，并把文档内容标记为数据而非系统指令。

Q54 为什么检索隔离很重要?

核心回答：检索系统必须先按用户、租户和文档权限过滤，再做语义召回。否则模型即使不直接展示原文，也可能通过总结或推断泄露无权访问的信息。

- 权限过滤应进入数据库或索引查询条件。
- 缓存、重排输入、trace 和引用页面同样要隔离。

Q55 什么是最小权限原则?

核心回答：每个 Agent 和工具只获得完成当前任务所需的最小数据范围、操作范围和有效期。读写工具分离，危险操作使用更窄的参数和额外审批。

- 避免把管理员凭据注入通用 Agent。
- 权限应在服务端校验，模型选择了工具并不代表已经授权。

Q56 哪些操作需要 Human in the Loop?

核心回答：高影响且难以撤销的动作应人工确认，例如发送外部消息、删除数据、执行资金操作、提交代码或扩大数据共享范围。确认页面要展示实际参数和影响，而不是模糊地问是否继续。

- 低风险只读操作可按策略自动批准。

- 审批必须绑定调用 ID、参数哈希和过期时间。

Q57 如何管理密钥和敏感信息?

核心回答：密钥放入密钥管理系统或环境注入，不进入提示词、代码仓库和普通日志；调用工具时由运行时附加凭据。个人信息按用途最小化收集，并提供保留期限和删除机制。

- 模型输出和异常栈也可能泄密，需要脱敏过滤。
- 对外部模型传输数据前要检查数据分类和供应商策略。

Q58 如何控制 Agent 的爆炸半径?

核心回答：使用沙箱、只读文件系统、网络域名白名单、数据库只读账户、资源配额和单任务预算。对写操作保留审计日志和补偿路径，并限制一次任务能影响的对象数量。

- 把所有工具放进一个高权限进程会放大单点失误。
- 安全控制要在模型之外执行，并用对抗测试验证。

十 系统设计题

系统设计回答建议按需求、数据流、关键组件、可靠性、安全、评估六部分展开。

设计一个职位匹配 Agent

- 需求：输入简历和求职偏好，返回候选岗位、匹配依据、技能差距与行动建议；所有结论可回溯。
- 数据层：简历解析为结构化字段，同时保留规范化原文和标题感知 Chunk；职位数据建立关键词索引、向量索引和权限元数据。
- 编排层：意图识别后进入检索、融合、重排、匹配分析、技能差距分析和报告生成节点；高影响改写或外部投递前触发人工确认。
- 检索层：关键词与向量并行召回，RRF 融合，交叉编码器或轻量模型重排；按地点、学历和岗位类别过滤。
- 可信层：证据片段携带 source_id 和位置；输出字段关联证据；无充分证据时降置信度或拒绝生成具体结论。
- 工程层：使用 FastAPI 提供接口，PostgreSQL 保存任务、状态和证据，SSE 推送步骤，队列与 Semaphore 控制并发，Docker 部署。
- 评估：离线统计 Recall@k、NDCG、引用准确率和匹配人工一致率；线上统计任务成功率、首 token 延迟、总成本与人工修改率。

设计一个长任务编程 Agent

- 把任务拆成分析、计划、执行、测试、审查和总结阶段，确定性检查由代码节点完成。
- 工具按文件读取、局部修改、测试和版本控制分组；写入操作限制工作目录并保留差异。
- 每步持久化状态和工具结果摘要，使用幂等操作和检查点支持进程故障后恢复。
- 长输出采用 SSE；客户端断开后按任务模式决定取消或转后台执行，并提供 task_id 查询进度。

- 限制最大步骤、总 token、并发工具数和命令运行时间；危险命令或外部发布必须人工确认。
- 测试包括工具契约、故障注入、取消传播、恢复一致性和任务成功率评估。

如何降低幻觉并提高可控性

- 将可确定的计算、过滤和权限判断移出模型。
- 用结构化输出和业务校验限制结果形态。
- 通过 RAG 或工具获取实时事实，并要求关键断言关联证据。
- 对缺少证据、工具失败和输入冲突定义明确的拒答与降级策略。
- 对模型版本、提示词、工具和索引建立回归评估与灰度发布。

如何支持高并发 Agent 请求

- API 层快速创建任务并返回 task_id，长任务进入队列；同步请求只保留明确的短时预算。
- 模型和外部工具分别设置并发池、速率限制、超时、熔断与降级。
- 可并行的只读工具使用 TaskGroup，写操作按资源键加锁或串行化。
- 状态写入使用事务、乐观锁或事件序列号，避免并发覆盖。
- 缓存稳定提示前缀、检索结果和确定性工具结果，但缓存键必须包含权限与版本。
- 监控队列时间、模型 P95、工具 P95、限流率、失败率和单任务成本，并据此扩容。

十一 项目深挖回答模板

以下表达严格依据简历已有内容整理。面试时补充真实负责范围、数据规模和验证结果，不要编造指标。

AI 求职规划与岗位匹配 Agent

项目介绍

这个项目面向求职规划和岗位匹配场景。我使用 LangChain 将职位检索、匹配分析和技能差距分析封装为结构化工具，再用 LangGraph 编排意图路由、任务计划、人工确认和恢复流程。数据侧同时保留简历与 JD 的结构化字段和规范化原文，并按标题感知方式切分。检索阶段结合全文检索、向量召回、RRF 融合和候选重排，最终输出带可验证引用的匹配依据。

为什么同时保留结构化字段和原文

结构化字段适合学历、技能、地点等精确过滤和打分，原文适合保留职责语境和项目细节。只用结构化字段会损失信息，只用原文又不利于稳定过滤，所以两种表示互补。

为什么选择 LangGraph

这个场景包含意图分流、多工具调用、人工确认和失败恢复，执行路径不是一条固定链。LangGraph 能把这些分支和循环显式表达为状态图，并通过 checkpoint 支持暂停和恢复。

如何处理 Prompt Injection

我把知识库内容视为不可信数据，在进入模型前做可疑指令扫描和检索隔离；工具权限和数据访问由代码控制；输出要求携带引用，使关键描述能回溯到原始材料。

MiniOpenClaw Agent 编程助手

项目介绍

这个项目是基于 Python 的 Agent 编程助手，支持 CLI 和飞书 IM 双入口。我把系统拆成模型接口层、Agent 编排层、应用能力层和消息渠道层，核心实现 ReAct 多轮循环、SSE 流式事件、工具串并行执行、异常降级和生命周期 Hook。为了支持长对话，还实现了上下文压缩、指数退避重试、JSONL 会话持久化和会话分叉结构。

四层架构的价值

模型接口层屏蔽不同模型差异，编排层管理状态与工具循环，应用能力层承载具体工具，渠道层适配 CLI 和飞书。这样更换模型或接入新渠道时，不需要改动核心 Agent 逻辑。

如何实现流式事件

我把 token、思考状态摘要、工具开始、工具结果、错误和完成定义为统一事件类型，由 Agent 编排层生成，再由 CLI 或飞书适配器消费。这样渠道层只负责展示和更新，不参与核心决策。

如何测试核心链路

我使用 Pytest 覆盖 SSE 事件顺序、Agent 工具闭环、上下文压缩、飞书消息更新和并发执行。测试时固定模型响应或使用 Fake Model，重点验证状态变化、工具参数、异常分支和终止条件。

面试官继续追问时要补充的真实信息

- 你的具体负责范围，哪些模块独立完成，哪些使用了现成组件。
- 语料或任务规模，包括文档数量、平均 Chunk 数和并发量。
- 关键参数如何选择，例如 top_k、RRF 的 k、重排候选数、压缩阈值和最大 Agent 步数。
- 遇到过的具体故障、定位过程、修复方案和验证方式。
- 可量化结果，例如召回率、任务成功率、延迟、测试数量或成本变化。没有测量的数据应直接说明尚未系统统计。

十二 高频速记清单与参考资料

最后一轮复习只看本页，确保每个概念都能用自己的项目举例。

面试前十分钟速记

- Agent = 模型决策 + 工具执行 + 状态更新 + 终止条件。
- 固定路径优先 Workflow，需要语义决策时再引入 Agent。
- 模型只能提出工具调用，运行时负责校验、授权、执行和审计。
- LangChain 偏高层组件，LangGraph 偏有状态编排和持久化运行时。
- Checkpoint 保存步骤状态，thread_id 标识连续会话；副作用必须幂等。
- RAG 分为离线索引和在线检索生成；召回与生成要分层评估。
- 混合检索兼顾语义改写和精确术语；RRF 按排名融合不同检索器。
- 短期记忆属于线程状态，长期记忆跨线程；不是所有历史都应进上下文。
- SSE 适合服务端单向流式推送；断连要向下游传播取消。
- 异步并发必须配合超时、Semaphore、有界队列和背压。
- MCP 统一连接上下文与工具，核心角色是 Host、Client、Server。
- Agent 评估同时看最终答案、单步决策和完整轨迹。
- Prompt Injection 不能只靠提示词解决，必须结合隔离、最小权限和人工确认。
- 回答系统设计题时覆盖需求、数据流、状态、可靠性、安全和评估。

反问面试官

- 团队当前的 Agent 更偏固定工作流还是开放式工具调用?
- 线上质量主要通过哪些离线评估集和业务指标衡量?
- 长任务状态、人工确认和失败恢复目前如何实现?
- 工具权限、知识库隔离和 Prompt Injection 防护由哪些层负责?
- 这个岗位入职前三个月最希望解决的工程问题是什么?

版本敏感提醒

Agent 框架 API 更新较快。面试回答应优先讲稳定概念、数据流和工程权衡；涉及 create_agent、状态类型、持久化适配器或 MCP 版本字段时，以项目实际版本和官方文档为准。

参考资料

- LangGraph Overview
- LangGraph Persistence
- LangChain Retrieval and RAG
- LangSmith Agent Evaluation
- Model Context Protocol Specification
- OWASP Top 10 for LLM Applications 2025
- ReAct Paper
- Python asyncio Tasks